

Actuate

Architecture Design Document v2

*Integrating Frontier VLA & Dexterous-Manipulation Research
into a Production Multimodal-to-Training-Ready Pipeline*

This is a **revision** of the v1 architecture, not a replacement. It integrates the current (2025–2026) state of the art in vision-language-action models and dexterous human-to-robot retargeting into the existing layer structure, and — per the review discipline used throughout this project — it is explicit about which parts are already built (§1–§11 plus the service and app layers), which need real rework, and which are net-new workstreams. The two capabilities foregrounded here are **dexterous finger-level retargeting** and **VLA-ready output representation**.

Contents

1	What Changed Since v1, and Why
2	The Field Has Consolidated: What "Training-Ready" Means in 2026
3	Research Integrated — Cluster by Cluster
4	Build Status: Additive Map Onto v1
5	Layer 2 Rework — Sensor-Fused Interaction (formalized)
6	Layer 3 Rework — Canonical Representation as a VLA-Native Schema
7	Layer 5 Expansion — Dexterous Retargeting Cascade
8	Layer 6 Rework — Task Field Alignment for VLA Training
9	Layer 7 Expansion — Flow-Matching / Action-Chunk-Ready Delivery
10	Honest Rework Ledger — What Existing Code Must Change
11	Sequenced Build Roadmap
12	References

1 What Changed Since v1, and Why

The v1 document specified a complete, layered pipeline (L0–L8) for turning multi-rig controlled capture into certified, retargeted robot data. That structure holds. What has changed is external: the downstream consumers of this data — the VLA models a customer actually trains — have consolidated around a specific architecture and a specific data contract over 2025–2026, and the dexterous-retargeting literature has matured enough that finger-level transfer is no longer a research gamble but a set of buildable, published methods with released code.

This revision does two things v1 did not. First, it makes the pipeline's **output** explicitly native to the flow-matching, action-chunked VLA training regime that $\pi 0/\pi 0.5$, GR00T N1.5, SmoVLA, and DM0 all now share — rather than treating "delivery format" as a generic export concern. Second, it upgrades the retargeting engine (v1 Layer 5) from arm/wrist-level functional retargeting to a **dexterous cascade** that handles finger-level, contact-rich transfer, drawing on GeoRT, DexMachina, and the retargeting-objectives ablation literature.

v1 answered "how do we build a correct, certified pipeline." v2 answers "how do we make its output the thing a frontier lab's training run consumes with zero adaptation, including for dexterous hands."

2 The Field Has Consolidated: What "Training-Ready" Means in 2026

A pipeline claiming "training-ready" output has to hit a moving target. As of 2026 that target has stopped moving in two important ways, which is good news — it means the output contract can be specified concretely rather than hedged.

2.1 Action representation has converged on flow matching

The dominant generalist-VLA architecture is now a vision-language backbone with a **flow-matching action expert** that generates continuous, action-chunked control. $\pi 0$ established this (SigLIP/PaliGemma-class VLM + flow-matching head producing smooth continuous actions), and $\pi 0.5$, GR00T N1.5, DM0, HiMoE-VLA, and ROCKET all follow the same pattern. The practical consequence for a data pipeline: continuous, temporally smooth, action-chunked trajectories are strictly more valuable than discrete tokenized actions, because they match what a flow-matching head is trained to predict, and are explicitly noted as better for contact-rich and dexterous tasks. Delivered actions must therefore be continuous, densely time-aligned to proprioception, and chunk-friendly — not sparse or tokenized.

2.2 The data contract has converged on LeRobot v3 / RLDS

The de-facto delivery formats are **LeRobot Dataset v3** (chunked Parquet for per-frame tabular data, chunked MP4 per camera, episode metadata tying them together) and **RLDS** (episodes as timestep sequences of observations, actions, language, metadata). Both expect the same core fields: `observation.images.*` (multi-view, consistent camera names), `observation.state` (proprioception), `action` (time-aligned control targets), and `task` (natural-language instruction — a required field most VLAs depend on). LeRobot v3 additionally supports action-chunking-native loading via delta-timestamp windows, which is why dense time-alignment between state and action is not optional.

2.3 Human video as a validated pretraining source

The premise underlying this whole pipeline is now externally validated: pretraining on 20,000+ hours of egocentric human video improved downstream robot task success by 54% over training from scratch (NVIDIA EgoScale), and co-training on human-hand demonstrations alongside robot data measurably improves manipulation — with one hour of human data reported as more valuable than one hour of additional robot data (EgoMimic). This is the market evidence that the human-video-to-robot-data category this pipeline targets is real, not speculative.

3 Research Integrated — Cluster by Cluster

The following clusters map directly onto pipeline layers. Each entry notes the specific technique adopted and where it lands.

3.1 VLA output architecture → Layers 6, 7

Source	What we take	Lands in
$\pi 0$ / $\pi 0.5$ (Physical Intelligence)	Flow-matching, action-chunked continuous action target; task field as required conditioning	L6, L7
GR00T N1.5, DM0, SmoIVLA, HiMoE-VLA	Confirmation that flow-matching + proprioception-conditioned action expert is the consolidated standard, not a single-lab bet	L7
VLAFlow / co-training studies	Language supervision + future-latent signals preserve generalization — so language grounding (L6) is training signal, not just documentation	L6
LeRobot v3 / RLDS specs	Concrete field schema and action-chunk-native, delta-timestamp loading contract	L7

3.2 Dexterous retargeting → Layer 5

Source	What we take	Lands in
GeoRT (Yin et al., 2025)	Fast, principled neural hand-retargeting for the finger branch	L5
DexMachina (Mandi et al., 2025)	Functional retargeting via virtual-object-controller curriculum; contact-guided, task-state-tracking objective for bimanual articulated tasks	L5
Retargeting-objectives ablation (Xin et al., 2025)	Empirical evidence of which retargeting objectives actually matter — informs IK/scoring cost design	L5
Learning to Transfer Hand Skills (Park et al., 2025)	Joint human-object-robot motion manifold; infer plausible robot action rather than pure kinematic copy	L5
ManipTrans / SPIDER	Residual-learning transfer and physics-informed scalable retargeting as validation/refinement references	L5

3.3 Capture interface → Layers 0, 3

Source	What we take	Lands in
DexUMI	Human-hand-as-UMI capture design for the dexterous/glove rig types	L0
EgoInfinity (v1 baseline)	Metric calibration, interaction-state refinement, SE(3) root-frame estimator — retained as arm-branch backbone	L1,L2,L5

Caveat carried from prior discussion: these are integrated from abstracts, released code, and the current literature — before committing engineering effort to any single method, the paper and its code must be read in full and checked against our actual rig/embodiment assumptions.

4 Build Status: Additive Map Onto v1

Per the "additive but flag rework" scope, every v2 change is positioned against what already exists. The figure below is the honest status map — most layers are built and need only extension; two need genuine rework; one (dexterous retargeting) is a net-new workstream.

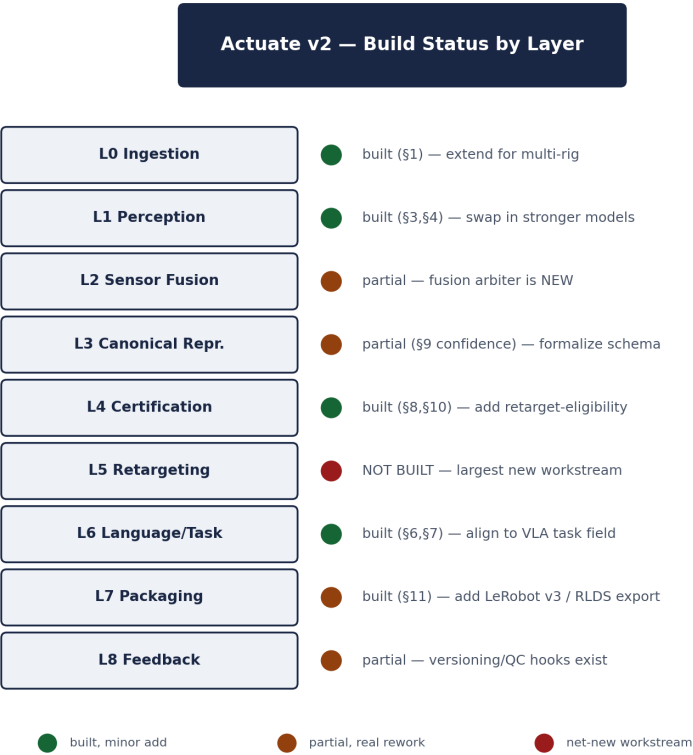


Figure 4.1 — v2 build status. Green: built, minor addition. Amber: partial, real rework required. Red: net-new workstream.

5 Layer 2 Rework — Sensor-Fused Interaction (formalized)

v1 introduced the sensor-fusion arbiter conceptually. v2 formalizes it, because the dexterous branch (Section 7) depends on trustworthy finger-contact state, and a vision-only grasp signal is not precise enough to drive finger-level retargeting.

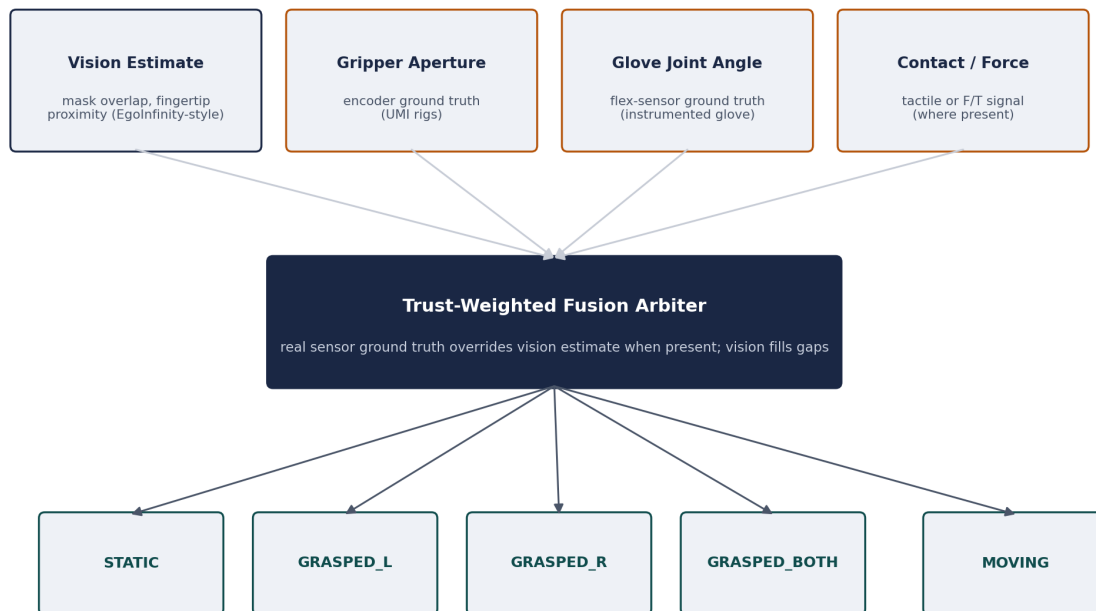


Figure 5.1 — The trust-weighted fusion arbiter (carried from v1), now the required upstream of dexterous retargeting.

5.1 Why this is now load-bearing, not optional

Finger-level retargeting is only as good as the contact/grasp ground truth feeding it. On instrumented-glove and UMI rigs, real joint-angle and aperture sensors give per-finger ground truth that a pure-vision grasp heuristic cannot match — and the dexterous methods in Section 7 (especially DexMachina's contact-guided objective) assume reliable contact state. The arbiter's trust ordering (hardware-measured > vision-primary > vision-fallback) is therefore promoted from a quality nicety to a hard prerequisite for an episode to be eligible for the dexterous branch at all.

Rework flag

The existing §5 primitive/confidence code computes grasp state but does not yet expose a clean, per-finger contact-confidence signal in the form the dexterous branch needs. This is a real (not cosmetic) extension of existing code, not a new module.

6 Layer 3 Rework — Canonical Representation as a VLA-Native Schema

v1's canonical representation was agent-agnostic and complete for retargeting. v2 additionally requires it to carry, per frame, exactly the fields a VLA training loader will later demand — so that packaging (Layer 7) is a pure format transcription, never a re-derivation.

6.1 Required per-frame fields (VLA-native)

Canonical field	Maps to VLA field	Notes
multi_view_frames	observation.images.*	Consistent camera naming enforced from ingestion
proprioceptive_state	observation.state	Joint/EE pose + gripper OR full finger-joint vector for dexterous rigs
control_target (continuous)	action	Dense, time-aligned to state; chunk-friendly; never tokenized
language_instruction	task	From L6; required, not optional
contact_state / finger_contact	(auxiliary training signal)	Enables contact-aware policies and dexterous filtering
per_field_confidence + provenance	(dataset stat / filter)	Lets customer filter by trust before training

Rework flag

v1's §9 confidence tree and §3-era canonical outputs cover part of this, but proprioceptive_state for dexterous rigs (full finger-joint vectors) and a formal continuous control_target field are additions. The schema must be frozen and versioned before Layer 7 export adapters are built against it.

7 Layer 5 Expansion — Dexterous Retargeting Cascade

This is the largest new workstream. v1's Layer 5 did arm/wrist functional retargeting well (SE(3)-equivariant root-frame estimation + IK, EgoInfinity-style) but treated fingers as a secondary geometric mapping. v2 replaces that with a two-branch cascade that treats arm and finger retargeting as separate, differently-constrained problems reconciled by contact consistency — the design pattern the current dexterous literature converges on.

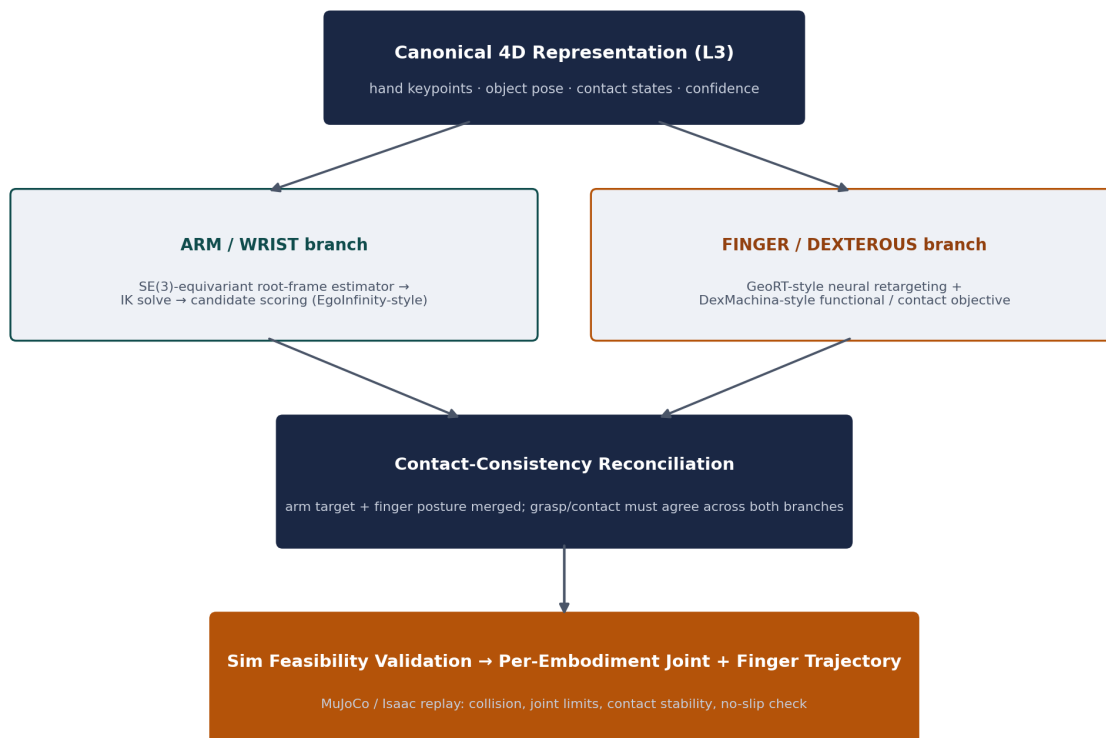


Figure 7.1 — Dexterous retargeting cascade: decoupled arm and finger branches reconciled by contact consistency, then sim-validated.

7.1 Arm/wrist branch (retained from v1)

Unchanged in principle: the SE(3)-equivariant, flow-matching root-frame estimator produces candidate root frames, IK solves per candidate, and candidates are scored on convergence, manipulability, joint-limit margin, and smoothness. This branch is already specified in v1 Section 9 and needs no rework beyond feeding the reconciliation step.

7.2 Finger/dexterous branch (new)

- **Neural finger retargeting (GeoRT-style)** — a fast, principled neural map from recovered hand keypoints (or real glove joint angles) to the target dexterous hand's joint configuration, replacing v1's geometry-only finger mapping.
- **Functional / contact objective (DexMachina-style)** — rather than copying finger pose exactly, optimize for reproducing the *task-relevant object-state outcome*, using contact positions as guidance. This is what makes transfer robust across hands with different finger counts and kinematics.
- **Objective weighting informed by ablation evidence** — the retargeting-objectives study gives empirical priors on which cost terms matter, so the finger-branch objective is tuned from evidence rather than guesswork.

7.3 Contact-consistency reconciliation (new)

The two branches can disagree — an arm-IK solution may place the wrist where the finger branch's grasp posture is infeasible. The reconciliation step enforces that arm target and finger posture agree at contact: grasp/contact frames must be mutually consistent, and disagreements beyond tolerance flag the episode for that embodiment rather than shipping a physically inconsistent trajectory.

7.4 Simulation feasibility (extended from v1)

v1's sim-replay validation is extended with dexterous-specific checks: contact stability and no-slip constraints, not just collision and joint limits. This directly implements the honest limitation EgoInfinity documents — that coarse grasp detection does not guarantee contact-level accuracy — by making contact correctness a validated, certified property rather than an assumed one.

8 Layer 6 Rework — Task Field Alignment for VLA Training

v1's language grounding (§6/§7, VLM-generated) produces good human-readable instructions. v2 reframes that output as **training signal**, because the current co-training literature shows language supervision measurably preserves a VLA's generalization — the task field is not documentation, it is a conditioning input the model trains against.

- The task field must be populated for every episode (VLAs depend on it; some loaders only offer a fragile prompt-fallback when it is missing).
- Instruction style should match training-time conditioning: concise, imperative, describing the task outcome — consistent with how VLA task prompts are phrased at inference.
- The open-vocabulary task identification already built (VLM cross-check) feeds this directly; the closed action-ontology (v1 §10.3) remains the primitive-level vocabulary beneath it.

Rework flag

Mostly alignment, not rebuild — existing VLM language grounding is reused, with output phrasing and required-field guarantees tightened to the VLA task-field contract.

9 Layer 7 Expansion — Flow-Matching / Action-Chunk-Ready Delivery

This is where the VLA-native canonical representation (Section 6) becomes an actual dataset a customer's training script loads unmodified.

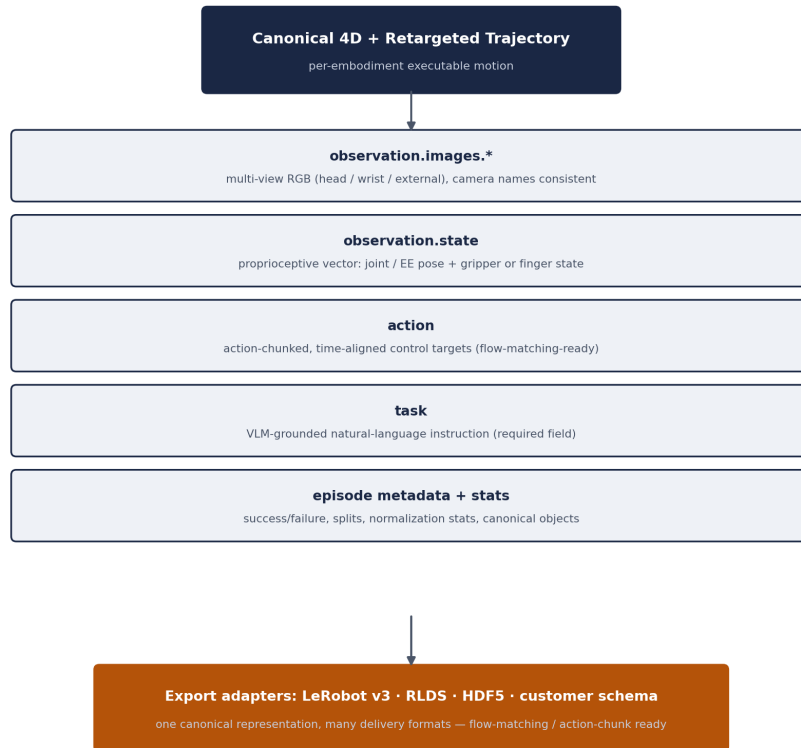


Figure 9.1 — VLA-ready output stack: the canonical representation maps field-for-field onto the LeRobot v3 / RLDS contract.

9.1 Delivery requirements made concrete

- **Continuous, action-chunked actions** — delivered actions are continuous and densely time-aligned to proprioception, so a flow-matching action head or an action-chunking loader consumes them natively.
- **LeRobot v3 primary, RLDS secondary** — chunked Parquet + per-camera chunked MP4 + tying metadata for LeRobot; timestep-sequence RLDS for Open-X-convention stacks; both from one canonical source.
- **Normalization statistics shipped** — per-dataset action/state normalization stats included in metadata, since VLA training expects them and omitting them forces customer-side recomputation.
- **Delta-timestamp-friendly layout** — episode/chunk structure supports action-chunk windowed loading without pathological repeated file opens.

Rework flag

v1's §11 packaging is real but format-generic. A concrete, tested LeRobot v3 exporter and an RLDS exporter are genuine additions — and, per prior review discipline, must be verified against a real VLA loader (e.g. LeRobot's own loader) rather than only asserted schema-correct.

10 Honest Rework Ledger — What Existing Code Must Change

Keeping to the review standard used throughout this project: the following is the explicit list of already-built pieces that this v2 requires changing, so nothing is quietly assumed compatible.

Existing piece	Change required	Severity
\$5 primitives / \$9 confidence	Expose per-finger contact-confidence signal for dexterous branch	Extension
\$3-era canonical representation	Add finger-joint proprioception + continuous control_target; freeze & version schema	Rework
Layer 5 (v1 \$9 retargeting)	Add finger branch + contact reconciliation + dexterous sim checks	Net-new
\$6/\$7 language grounding	Tighten to VLA task-field contract; guarantee field presence	Alignment
\$11 packaging	Add real LeRobot v3 + RLDS exporters, verified against a real loader	Net-new export
FastAPI service / app	Surface retargeting-eligibility + finger-contact in status/results; expose new export formats	Extension
Task taxonomy (open-vocab work)	Feed task field as training signal; keep closed action ontology beneath	Alignment

11 Sequenced Build Roadmap

Ordered so each step unblocks the next and surfaces risk early — the same principle used across the v1 build (do the load-bearing, hardest-to-validate thing before the things that depend on it).

Phase 1 — Freeze the VLA-native schema (Layer 3)

Before any exporter or dexterous work, formalize and version the canonical per-frame schema (Section 6). Everything downstream is built against it; changing it later is the expensive mistake to avoid. Low compute, high leverage — do it first.

Phase 2 — LeRobot v3 / RLDS exporters (Layer 7)

Build and verify the exporters against a real LeRobot loader on already-processed (arm/gripper) data. This makes the pipeline deliver genuinely training-ready output for non-dexterous embodiments immediately, independent of the harder dexterous work — a shippable milestone.

Phase 3 — Formalize sensor fusion for finger contact (Layer 2)

Extend the fusion arbiter to expose per-finger contact confidence (Section 5). Prerequisite for the dexterous branch; buildable without new hardware or GPU-heavy models.

Phase 4 — Dexterous retargeting cascade (Layer 5)

The big one. Prototype the finger branch against a released method with public code (GeoRT and DexMachina are the two with usable code) on one target dexterous hand, validate in simulation with contact/no-slip checks, then generalize. This needs GPU and is the highest-risk workstream — surface its problems while the rest of the pipeline is already delivering value.

Phase 5 — Task-field alignment + delivery hardening (Layers 6, 7)

Tighten language grounding to the VLA task contract and ship normalization stats + action-chunk-friendly layout. Largely alignment on top of proven components.

Phase 2 is deliberately early and independent of the dexterous work: it means the pipeline can deliver genuinely VLA-training-ready gripper/arm data to a customer well before the dexterous cascade is finished — value shipped, not gated behind the hardest component.

12 References

- Wang et al. EgoInfinity: A Web-Scale 4D Hand-Object Interaction Data Engine. arXiv:2606.17385, 2026.
- Black et al. $\pi 0$: A Vision-Language-Action Flow Model for General Robot Control. Physical Intelligence, 2024.
- Yin et al. Geometric Retargeting (GeoRT): A Principled, Ultrafast Neural Hand Retargeting Algorithm. arXiv:2503.07541, 2025.
- Mandi et al. DexMachina: Functional Retargeting for Bimanual Dexterous Manipulation. arXiv:2505.24853, 2025.
- Xin et al. Analyzing Key Objectives in Human-to-Robot Retargeting for Dexterous Manipulation. arXiv:2506.09384, 2025.
- Park et al. Learning to Transfer Human Hand Skills for Robot Manipulations. arXiv:2501.04169, 2025.
- DexUMI: Using the Human Hand as the Universal Manipulation Interface for Dexterous Manipulation. arXiv:2505.21864, 2025.
- Li et al. ManipTrans: Efficient Dexterous Bimanual Manipulation Transfer via Residual Learning. arXiv:2503.21860, 2025.
- SPIDER: Scalable Physics-Informed Dexterous Retargeting. arXiv:2511.09484, 2025.
- An et al. Dexterous Manipulation through Imitation Learning: A Survey. arXiv:2504.03515, 2025.
- VLAFlow: A Unified Training Framework for VLA Models via Co-training and Future Latent Alignment. arXiv:2607.01586, 2026.
- LeRobot v0.4.0 / Dataset v3.0 release notes, Hugging Face, 2025–2026.
- NVIDIA EgoScale; EgoMimic — egocentric-video pretraining and human-robot co-training results, 2025–2026.

This v2 document is additive to v1 and subject to the same discipline applied throughout the Actuate build: every integrated technique is a design intention to be validated against real code and real data before it is treated as done. Nothing here is launch-ready by virtue of being written down.